Teaching the expert cache when to fetch

Three single-change variants (V1, V2, V3) of the reuse-aware fetch rule, tested on gpt-oss-20b on an RTX 4070. Written for readers new to the project: from the setup to the final results.

In one sentence: V1, which replaces the rule's fixed cost assumptions with costs measured live while the model runs, made decoding **1.056x faster** than the original rule (95% range 1.016-1.096). V2 made it slower when the cache was small, and V3 was not reliably different. V1 is the only variant that passed the test we set before the run.

Item	Value
GPU / host	NVIDIA RTX 4070 12 GB (PCIe gen4 x4); Ryzen 5 8500G (6 cores); 30 GB RAM
Model	gpt-oss-20b (mixture of experts: 24 layers, 32 experts per layer, 4 used per token)
Run	results/reuse_aware_v2/pilot_20261020T140940Z (144 episodes, 20 Oct 2026)
New code	experiments/runtime_q/instrumentation/reuse_aware_v2_hook.py; driver options in experiments/runtime_q/driver.py; plan configs/runtime_q/reuse_aware_v2/gpt_oss_pilot.json; analysis experiments/runtime_q/analyze_reuse_aware_v2.py
Credits	Runtime-q harness and CPU/GPU split policy: Saketh. Reuse-aware v2 pilot: Ankit.

1. Background for beginners

1.1 What is a mixture-of-experts model?

A normal language model sends every word (token) through the same big block of weights in each layer. A **mixture-of-experts (MoE)** model splits that block into many smaller blocks called **experts**. In every layer a small **router** picks a few experts for the current token and only those run. gpt-oss-20b has 24 layers with 32 experts each, and the router picks 4 per layer. So per token, 24 x 4 = 96 expert runs happen, out of 768 experts in total.

1.2 Why do experts live on two devices?

The whole model does not fit in the 12 GB of GPU memory. FreeToken keeps all experts in normal (CPU) memory and keeps a **cache** of some experts on the GPU. The **cache rate** is the share of all experts the GPU cache can hold. We tested 0.10 (about 77 of 768 experts) and 0.40 (about 307).

- **Hit:** the chosen expert is already in the GPU cache. It runs on the GPU and costs almost nothing extra.
- **Miss:** the expert is not on the GPU. Call the number of misses in a layer **m**. Each miss can be handled in one of two ways.
- **Fetch:** copy the expert over the PCIe cable to the GPU (about 1.8 ms each on this machine) and run it there. It then stays in the cache, pushing out (**evicting**) the least-recently-used expert.
- **Run on CPU:** compute the expert on the CPU (about 0.4 ms each). Nothing is cached.

Both paths run **at the same time**: the GPU branch copies and runs its experts while the CPU branch runs the rest. The layer is done when the slower branch is done. Each layer must therefore answer one question: **of the m misses, how many (q) should be fetched to the GPU?** Figure 1 shows this.

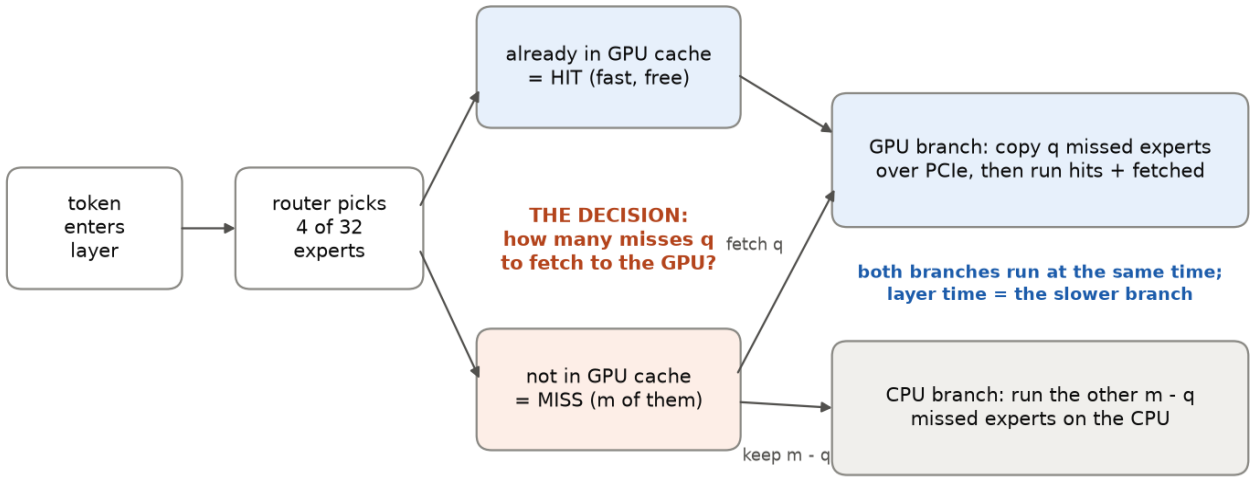


Figure 1. One MoE layer during decoding. The policy only chooses q, the number of missed experts copied to the GPU; the rest run on the CPU.

1.3 The rules we compare

- **q* (paper):** the FreeToken paper's rule. It picks q so the two branches finish at about the same time, using a fixed bandwidth ratio rho = 6.7 / 31.76 = 0.211 measured once offline. It ignores whether a fetched expert will be used again.
- **q* (built-in):** the same idea with the engine's own built-in fraction (about 0.17). On gpt-oss it rounds to q = 0 whenever m is 4 or less, so it **never fetches** and the GPU cache stays empty.
- **Original reuse-aware rule:** from the earlier runtime-q study. A fetch is also an investment: the expert stays on the GPU, and if the router picks it again on later tokens those are free hits. It fetches an expert only if the expected future hits pay for the extra time the fetch adds now. In the completed cross-model study it was 1.15x faster than paper q* on gpt-oss.

2. The original rule and its formula

For each missed expert, the rule estimates two things from information it already has:

- **p**: how likely this expert is to be picked again on a future token, from how often this layer picked it in the last 32 tokens (smoothed with a small prior: p = (count + 8 x 4/32) / (32 + 8)).
- **T**: how many tokens it will probably stay in the cache. If a slot is free, T = 32. Otherwise T = the age of the expert that would be evicted (least-recently used).

So **p x T** is the expected number of future hits from fetching it. Experts are considered in order of decreasing p, and the rule keeps fetching while:

```
p x T x v   >=   cost(q+1) - cost(q)

original:  cost(q) = max(q / rho, m - q)     (in 'CPU-expert' units)
           v = 1                             (one hit saves one CPU expert)
```

In words: the left side is the expected time saved later and the right side is the extra time the fetch adds to this layer now. q/rho is the GPU branch (each fetch costs 1/rho = 4.7 CPU-expert times) and m - q is the CPU branch. Both costs are assumptions fixed before the run.

3. What we changed: V1, V2 and V3

A diagnosis of the earlier study's logs (called Stage A) found three places where the formula's assumptions could be wrong. We built one variant per suspected problem. **Each variant changes exactly one thing**, so if it changes the speed we know why. All three live in a new file; the original rule is untouched.

Variant	What it changes	New formula piece	Stage A expected
V1 measured cost	Replaces the fixed cost model with times measured while the model runs	cost(q) = max(g0 + q x tp, c0 + (m-q) x tc) in ms; v = measured ms one hit saves	slower
V2 calibrated reuse	Corrects p x T by how often fetched experts were really reused so far	p x T becomes alpha x p x T, alpha = (real hits + 8) / (predicted hits + 8)	faster
V3 eviction-aware	Counts the reuse lost by evicting the victim expert	p becomes p - p_victim	slower

Table 1. The three variants. Every new quantity is computed only from what has already happened in the current run. Nothing is trained and no future information is used.

3.1 V1: measure the costs instead of assuming them

V1 times both branches in every layer and fits a straight line to each, separately for every run:

- **GPU branch:** time = g0 + tp x (number fetched). tp is the time per PCIe copy and g0 is the fixed GPU work (running the hits).
- **CPU branch:** time = c0 + tc x (number run on CPU). tc is the time per CPU expert.
- **v** = how many ms one future hit really saves. A hit removes one expert from the CPU branch, but that only helps if the CPU branch is the slower one, so v is often less than tc.
- Until a line has 8 measurements it uses the offline bandwidth numbers. The fit restarts for every run.

Worked example (numbers close to cache 0.10): m = 3 misses, best candidate p = 0.3, T = 7 tokens.

```
Original: gain = p x T = 0.3 x 7 = 2.10 hits;  cost = 1.74 CPU-experts
          gain 2.10 >= cost 1.74  ->  FETCH
V1:       gain = p x T x v = 0.3 x 7 x 0.37 = 0.78 ms
          cost = max(0.4+1.75, 0.02+2x0.4) - max(0.4, 0.02+3x0.4)
               = 2.15 - 1.22 = 0.93 ms
          gain 0.78 < cost 0.93  ->  do not fetch
```

The original rule says a hit is worth a whole CPU expert and leaves out the GPU's fixed work, so it judges this fetch worth it. V1 sees that a hit saves only about 0.37 ms and that the fetch makes the GPU branch the slow one, so it keeps the expert on the CPU.

3.2 V2: correct the reuse estimate with what really happened

Every time an expert that was fetched gets evicted, V2 compares how many hits it predicted (p x T at fetch time) with how many hits that expert really got while cached. alpha is the ratio of the totals so far, with +8 on both sides so it starts at 1. If alpha = 2, experts were reused twice as much as predicted, so every gain is doubled. Example: with alpha = 2.4 the gain 2.10 becomes 5.04, so V2 fetches more often.

3.3 V3: count the expert you throw away

When the cache is full, a fetch evicts the least-recently-used expert. That expert might have been reused too. V3 predicts which expert will be evicted (the same choice the engine makes) and uses p - p_victim instead of p. Example: p = 0.3 and p_victim = 0.2 leaves only 0.10, so the gain shrinks from 2.10 to 0.70.

4. How the experiment was set up

4.1 What a run (episode) is

One **episode** = start from an empty GPU cache, generate one warm-up answer, then generate the measured answer: 128 tokens with greedy decoding (always the most likely token, so runs are repeatable). We report **ms per token** during decoding: lower is better.

Setting	Value	Why
Cache rates	0.10 and 0.40	a small and a large GPU cache
Tasks	code (write a function that merges two sorted event streams); math (closed form of a(n+1) = 3a(n) + 1)	two kinds of text that use experts differently
Repeats	4 per policy and cell: rounds 0-2 timed, round 3 only for the detailed logs	writing logs slows decoding, so logged runs are never timed
Order	rotated each round	no policy always runs first or last
Burn-in	3 discarded episodes after each server start	the first episodes are slow on this host
Engine	FreeToken hybrid backend, eager mode (no CUDA graphs), LRU expert cache, 5 CPU threads, overlap scheduling off	same settings as the completed cross-model study
Policies	paper q*, built-in q*, original rule, V1, V2, V3 (plus a copy of the original rule on the old hook, and four overhead-check arms)	see Section 4.3

Table 2. Experiment settings.

4.2 How the code decides

FreeToken lets an outside Python file (a **hook**) choose q in every layer. The new hook reads the router's choices and the cache contents, computes the features above, and calls the chosen policy. The test driver starts one server per cache rate and task, and tells the hook which policy to use for each episode through a small control file. The driver gained three optional settings, so older plans still run exactly as before:

- **hook**: which hook file a server uses, or 'none' for a plain server;
- **log_rep**: which round writes the detailed decision log (here round 3 only);
- **control**: extra settings for the hook, such as the 'lean' feature set.

The new hook has a **lean** mode that gathers only what the reuse rules need. All six main policies run on the same lean hook, so they pay the same measurement cost and the comparison is fair.

4.3 Checking the cost of measuring

To measure branch times, the hook makes the CPU wait for the GPU twice per layer (a **sync**). To see what that costs, the code task also ran built-in q* on a plain server with no hook, and on the hook with progressively more switched on. Built-in q* never fetches on gpt-oss, so its cache stays empty either way and the comparison is clean.

4.4 How we judge results

- **Paired comparison:** in each round, each variant is compared with the original rule from the same round, cache rate and task. Speedup = original ms/token divided by variant ms/token, so above 1 means the variant is faster.

- **Geometric mean** of those ratios, with a **95% range** (confidence interval). If the whole range is above 1, the variant is reliably faster; if it includes 1, we cannot tell.
- **p-value:** roughly, the chance of a difference this large if there were really none. Below 0.05 counts as reliable.
- **Pass rule set before the run:** a variant advances only if it is not reliably slower at either cache rate and is reliably faster at one or more.

5. Results

5.1 Speed of every policy

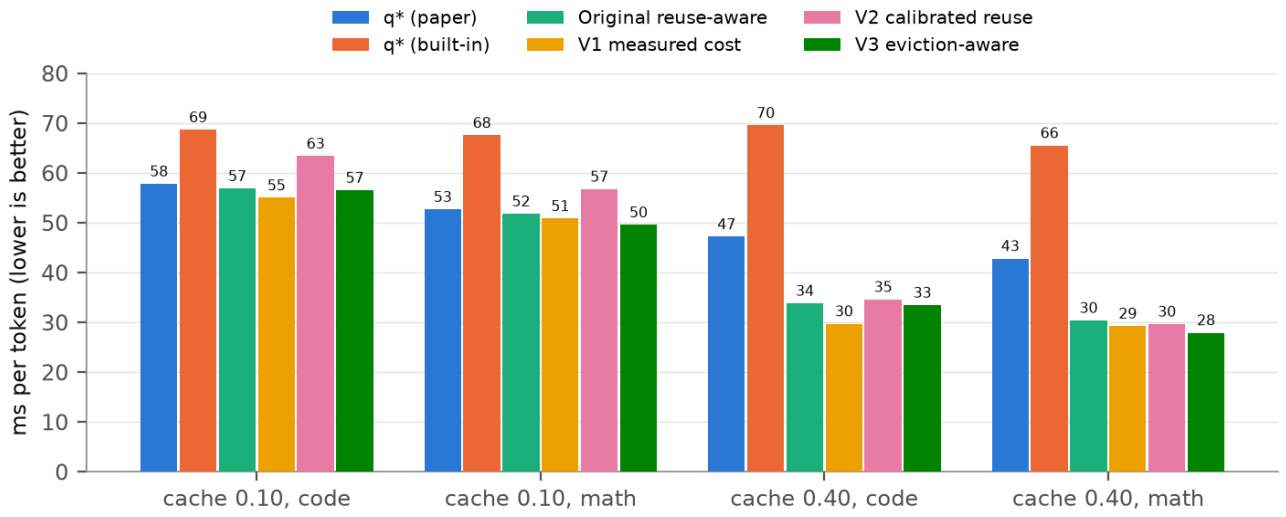


Figure 2. Mean ms per token over rounds 0-2 for each cache rate and task. Lower is better.

Cache, task	q* (paper)	q* (built-in)	Original reuse-aware	V1 measured cost	V2 calibrated reuse	V3 eviction-aware
0.10, code	57.8 ± 0.0	68.7 ± 1.7	56.9 ± 2.3	**55.0** ± 1.7	63.4 ± 0.9	56.5 ± 0.7
0.10, math	52.7 ± 2.2	67.6 ± 1.7	51.8 ± 2.9	50.9 ± 3.3	56.8 ± 2.6	**49.6** ± 3.3
0.40, code	47.2 ± 1.6	69.6 ± 1.9	33.9 ± 2.8	**29.8** ± 0.2	34.6 ± 2.9	33.5 ± 3.3
0.40, math	42.8 ± 0.2	65.5 ± 0.2	30.4 ± 2.9	29.3 ± 2.7	29.7 ± 2.9	**27.9** ± 0.0

Table 3. Mean ± standard deviation of ms per token (3 rounds). Bold = fastest in the row.

Two things stand out. With the larger cache (0.40), every reuse-aware rule is far faster than q*, because fetched experts keep paying off. Built-in q* is slowest everywhere, because it never fetches.

5.2 Each variant against the original rule

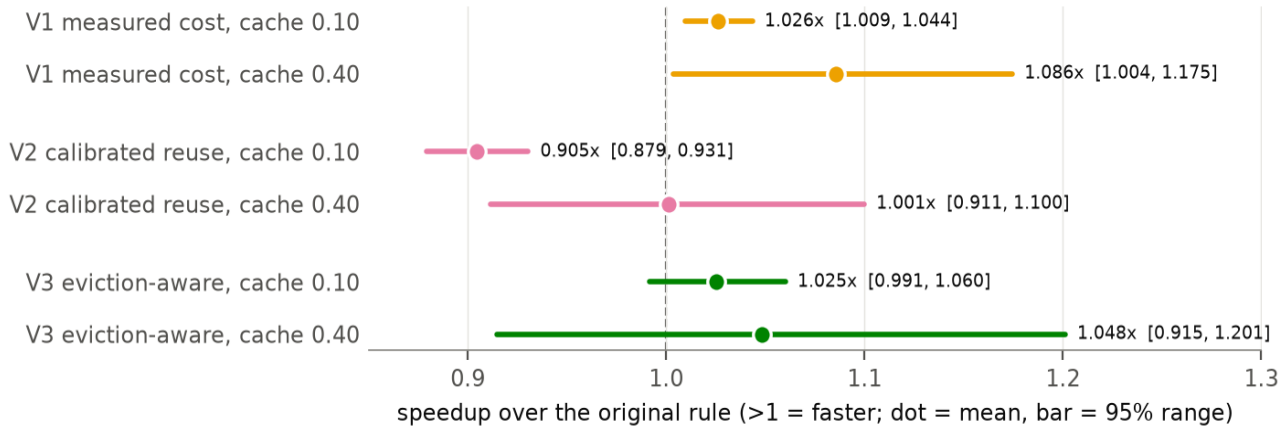


Figure 3. Paired speedup of each variant over the original reuse-aware rule (6 pairs per row: 2 tasks x 3 rounds). Dashed line = no difference.

Variant	All (12 pairs)	Cache 0.10	Cache 0.40	Reliably slower anywhere?	Reliably faster anywhere?	Advances?
V1 measured cost	1.056x [1.016, 1.096]	1.026x [1.009, 1.044]	1.086x [1.004, 1.175]	no	yes	**YES**
V2 calibrated reuse	0.952x [0.903, 1.003]	0.905x [0.879, 0.931]	1.001x [0.911, 1.100]	yes	no	no
V3 eviction-aware	1.037x [0.978, 1.098]	1.025x [0.991, 1.060]	1.048x [0.915, 1.201]	no	no	no

Table 4. Speedup over the original rule with 95% ranges. Highlighted = passed the rule set before the run.

5.3 Against the paper's q*

Policy	Cache 0.10	Cache 0.40
Original reuse-aware	1.018x [0.988, 1.048]	1.405x [1.312, 1.504]
V1 measured cost	1.044x [1.011, 1.079]	1.525x [1.412, 1.648]
V2 calibrated reuse	0.920x [0.907, 0.934]	1.407x [1.282, 1.543]
V3 eviction-aware	1.043x [1.010, 1.078]	1.472x [1.333, 1.627]

Table 5. Paired speedup over paper q* (6 pairs per cell). V1 widens the original rule's lead at both cache sizes.

6. Why each variant behaved as it did

Round 3 logged every decision in every layer. Figure 4 and Table 6 show what each rule actually did.

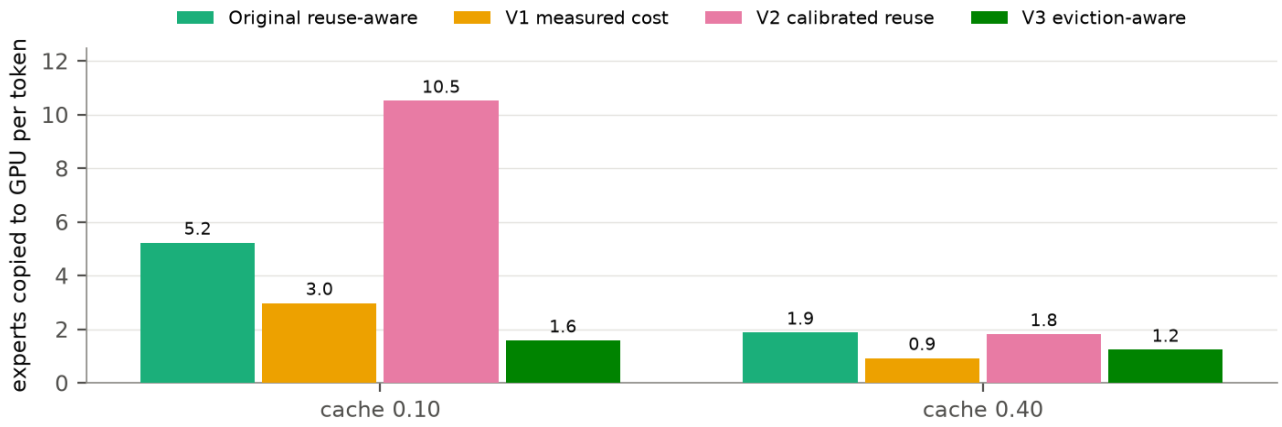


Figure 4. Experts copied to the GPU per token (all 24 layers), averaged over the two tasks.

Policy	Cache	Misses / token	Fetches / token	Hit rate	Evictions / token	MoE time / token (ms)
Original reuse-aware	0.10	60.9	5.2	0.366	4.7	38.5
Original reuse-aware	0.40	16.4	1.9	0.830	1.4	20.8
V1 measured cost	0.10	60.2	3.0	0.373	2.5	37.5
V1 measured cost	0.40	16.7	0.9	0.827	0.4	19.7
V2 calibrated reuse	0.10	60.2	10.5	0.373	10.0	42.8
V2 calibrated reuse	0.40	15.7	1.8	0.837	1.3	20.1
V3 eviction-aware	0.10	60.5	1.6	0.370	1.1	35.3
V3 eviction-aware	0.40	16.2	1.2	0.831	0.7	19.8

Table 6. Decision-log summary (round 3, averaged over the two tasks). Hit rate = share of the 96 expert uses per token that were already on the GPU.

6.1 V1: fewer fetches, same hits, less time

V1 fetched about 40-50% fewer experts than the original rule (3.0 vs 5.2 per token at cache 0.10, 0.9 vs 1.9 at 0.40), yet its hit rate stayed about the same. The fetches it skipped were ones that did not pay off. The measured costs explain why:

Quantity	Measured range	Meaning
tp	1.74-1.92 ms	one PCIe copy plus its GPU work
tc	0.39-0.42 ms	one expert on the CPU
g0	0.24-0.41 ms (when fetching)	fixed GPU work per layer
v	0.22-0.38 ms	real saving of one future hit

Table 7. Fitted costs at the end of the logged runs. A fetch costs about 1.8 ms while a hit saves about 0.22-0.39 ms, so a fetch has to be reused about 5-8 times to pay off.

The copy time itself is close to what the paper's ratio implies (1/rho = 4.7 CPU experts vs measured tp/tc of about 4.5). The difference is the value of a hit. The original rule counts each hit as one full CPU expert (0.40 ms), but a hit only saves time when the CPU branch is the slow one. With the larger cache there are few misses, the CPU branch is short, and v drops to about 0.22-0.26 ms. V1 sees this and fetches less, which is why its gain is largest at cache 0.40.

6.2 V2: more fetches, no more hits

alpha ended between 1.32 and 3.33: fetched experts did get more hits than predicted. So V2 fetched much more (10.5 vs 5.2 per token at cache 0.10), but the hit rate did not move (0.373 vs 0.366). Every extra fetch evicted another expert that would also have been reused, so the hits it gained were hits lost elsewhere. alpha measures the hits a fetched expert receives, not the net gain after the victim's lost hits. Stage A's conclusion that the rule fetches too little is therefore wrong.

6.3 V3: right idea, too small an effect to see

V3 cut fetches sharply at cache 0.10 (1.6 vs 5.2 per token) with about the same hit rate. It was faster on average at both cache rates (1.025x and 1.048x), but the 95% ranges include 1. With only 3 rounds we cannot tell it apart from no change. The victim prediction it relies on was correct for 99.5-100% of actual evictions.

6.4 Checks that the code did what it should

- Each variant's actual decisions matched its own formula, recomputed from the logged state, in 100% of logged layer decisions.
- The predicted eviction victim matched the engine's real eviction in 98.1-100% of cases.
- All episodes generated the full 127 decode steps.
- Code outputs were identical across all policies. Math outputs had 4-5 versions per cell, because an expert run on the CPU gives very slightly different numbers than on the GPU, which can change a later word. The completed study saw the same. V1 can also differ between its own rounds, because its decisions depend on measured times.
- The original rule on the lean hook and on the old study hook ran at the same speed (1.002x), so the new hook did not change the original rule's behaviour.

7. The cost of measuring

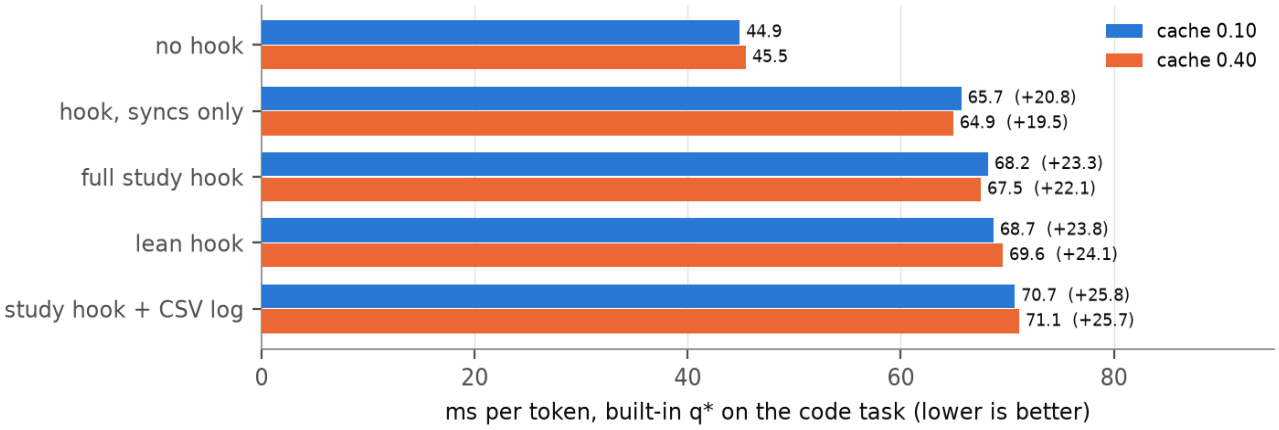


Figure 5. Built-in q* on the code task with different amounts of measurement switched on. Numbers in brackets are the extra ms per token over the plain server.

The plain server decodes at about 45 ms per token. Just the two syncs per layer that the timers need add about 20 ms per token (+45%). Choosing q itself takes about 4-5 ms per token, and CSV logging adds about 2-3 ms more. The lean feature set saved nothing on gpt-oss, because its cache is small and was already cheap to read. It may matter on models with larger caches.

What this means: every speed in this report, and in the completed cross-model study, was measured with this overhead included. V1 needs the branch timers to work, so we do not yet know how big its gain is without the syncs. That requires a version that reads the timers one layer late instead of waiting for them, which has not been built.

8. Conclusions and next steps

Variant	Result	Decision
V1 measured cost	1.056x faster than the original rule; reliably faster at both cache rates; 1.53x over paper q* at cache 0.40	**Advances**
V2 calibrated reuse	0.905x (slower) at cache 0.10, no change at 0.40	Rejected; its premise was wrong
V3 eviction-aware	1.037x on average, not reliably different	Does not advance; could be combined with V1 later

Table 8. Summary.

- **Main lesson:** the original rule fetched too much, not too little. The fix that worked was measuring what a fetch and a hit really cost on this machine.
- **Limits:** one model, one GPU, two tasks, 3 timed rounds per cell (spread up to about 3 ms per token at cache 0.40). All speeds include the hook's measurement overhead.
- **Proposed next step (not started, needs approval):** fix V1 exactly as tested and check it against the original rule and paper q* on Qwen and Gemma on the same RTX 4070, with more rounds for Qwen, where the possible gain is small.
- Possible later work: a version that reads timers without syncing; testing V1 + V3 together; making the driver avoid a port clash that stopped the run once (the run was resumed and lost no data).

How to reproduce

```
uv run --no-sync python -m experiments.runtime_q.analyze_reuse_aware_v2 \
    results/reuse_aware_v2/pilot_20261020T140940Z
# re-run (wraps scripts/run.sh runtime_q):
#   results/reuse_aware_v2/pilot_20261020T140940Z/run_plan.sh
```